EXPT -8 : Deploying a Fine-Tuned Language Model for Customer Support with Evaluation (BLEU & ROUGE)

AIM:

Develop a Python-based application to fine-tune and deploy a language model for customer support using platforms like Hugging Face Spaces or Streamlit, and evaluate its performance on the entire test dataset using BLEU and ROUGE metrics.

CODE:

train.py:

```
# ========================
# EX 8: Fine-Tuned Customer Support LM with BLEU & ROUGE
# ========================


# Install once
# !pip install -q transformers datasets evaluate rouge_score sacrebleu sentencepiece
accelerate streamlit


import os
import re
import numpy as np
import pandas as pd
from datasets import load_dataset
import evaluate


from transformers import (
   AutoTokenizer,
   AutoModelForSeq2SeqLM,
   DataCollatorForSeq2Seq,
```

```
    Seq2SeqTrainingArguments,
    Seq2SeqTrainer
)

# -------------------------
# 1. Load Dataset
# -------------------------
# Dataset has fields: question, answer
dataset = load_dataset("MakTek/Customer_support_faqs_dataset")

# Convert to one dataset split if needed
if "train" in dataset:
    full_data = dataset["train"]
else:
    # fallback if dataset loads differently
    full_data = list(dataset.values())[0]

# -------------------------
# 2. Preprocessing
# -------------------------
def clean_text(text):
    text = str(text).strip().lower()
    text = re.sub(r"\s+", " ", text)
    return text

def preprocess_examples(example):
    return {
        "input_text": "customer_query: " + clean_text(example["question"]),
```

```
        "target_text": clean_text(example["answer"])
    }

full_data = full_data.map(preprocess_examples)

# -------------------------
# 3. Train-Test Split
# -------------------------
split_data = full_data.train_test_split(test_size=0.2, seed=42)
train_data = split_data["train"]
test_data = split_data["test"]

print("Train size:", len(train_data))
print("Test size :", len(test_data))

# -------------------------
# 4. Load Pretrained Model
# -------------------------
model_name = "t5-small"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForSeq2SeqLM.from_pretrained(model_name)

max_input_len = 64
max_target_len = 96

# -------------------------
# 5. Tokenization
# -------------------------
```

```python
def tokenize_function(examples):
    model_inputs = tokenizer(
        examples["input_text"],
        max_length=max_input_len,
        truncation=True,
        padding="max_length"
    )

    labels = tokenizer(
        text_target=examples["target_text"],
        max_length=max_target_len,
        truncation=True,
        padding="max_length"
    )

    model_inputs["labels"] = labels["input_ids"]
    return model_inputs

tokenized_train = train_data.map(tokenize_function, batched=True)
tokenized_test = test_data.map(tokenize_function, batched=True)

# Remove extra columns to avoid trainer warnings
keep_cols = ["input_ids", "attention_mask", "labels"]
tokenized_train = tokenized_train.remove_columns(
    [col for col in tokenized_train.column_names if col not in keep_cols]
)
tokenized_test = tokenized_test.remove_columns(
    [col for col in tokenized_test.column_names if col not in keep_cols]
```

```
)


data_collator = DataCollatorForSeq2Seq(tokenizer=tokenizer, model=model)


# -------------------------
# 6. Fine-Tune Model
# -------------------------
training_args = Seq2SeqTrainingArguments(
    output_dir="./customer_support_t5",
    eval_strategy="epoch",
    save_strategy="epoch",
    learning_rate=2e-4,
    per_device_train_batch_size=8,
    per_device_eval_batch_size=8,
    weight_decay=0.01,
    num_train_epochs=8,
    predict_with_generate=True,
    logging_steps=10,
    save_total_limit=2,
    report_to="none",
    fp16=False
)


trainer = Seq2SeqTrainer(
    model=model,
    args=training_args,
    train_dataset=tokenized_train,
    eval_dataset=tokenized_test,
```

```
    tokenizer=tokenizer,
    data_collator=data_collator
)

trainer.train()

# -------------------------
# 7. Save Fine-Tuned Model
# -------------------------
save_path = "./final_customer_support_model"
trainer.save_model(save_path)
tokenizer.save_pretrained(save_path)
print(f"Model saved to: {save_path}")

# -------------------------
# 8. Inference Function
# -------------------------
import torch

device = torch.device("cpu")  # FORCE CPU (fix for Mac MPS issue)
model.to(device)

def generate_response(query, max_new_tokens=64):
    query = "customer_query: " + clean_text(query)

    inputs = tokenizer(
        query,
        return_tensors="pt",
```

```
        truncation=True,
        max_length=max_input_len
    )

    inputs = {k: v.to(device) for k, v in inputs.items()}

    with torch.no_grad():
        outputs = model.generate(
            **inputs,
            max_new_tokens=max_new_tokens,
            num_beams=4,
            early_stopping=True
        )

    return tokenizer.decode(outputs[0], skip_special_tokens=True)
# ------------------------
# 9. Generate Predictions on Test Dataset
# ------------------------
references = []
predictions = []
test_queries = []

for item in test_data:
    q = item["question"]
    ref = clean_text(item["answer"])
    pred = generate_response(q)

    test_queries.append(q)
```

```
    references.append(ref)
    predictions.append(clean_text(pred))

# Save sample predictions
results_df = pd.DataFrame({
    "Query": test_queries,
    "Reference Answer": references,
    "Generated Answer": predictions
})

print("\nSample Predictions:")
print(results_df.head(10))

results_df.to_csv("test_predictions.csv", index=False)

# ------------------------
# 10 & 11. Compute BLEU and ROUGE
# ------------------------
rouge = evaluate.load("rouge")
bleu = evaluate.load("bleu")

# ROUGE overall
rouge_scores = rouge.compute(predictions=predictions, references=references)

# BLEU-1 to BLEU-4
# evaluate bleu expects references as list of list of refs
refs_for_bleu = [[r] for r in references]
```

```
bleu1 = bleu.compute(predictions=predictions, references=refs_for_bleu, max_order=1)

bleu2 = bleu.compute(predictions=predictions, references=refs_for_bleu, max_order=2)

bleu3 = bleu.compute(predictions=predictions, references=refs_for_bleu, max_order=3)

bleu4 = bleu.compute(predictions=predictions, references=refs_for_bleu, max_order=4)


# -------------------------

# 12 & 13. Aggregate Evaluation Metrics

# -------------------------

metrics_table = pd.DataFrame({

   "Metric": ["BLEU-1", "BLEU-2", "BLEU-3", "BLEU-4", "ROUGE-1", "ROUGE-2",
"ROUGE-L"],

   "Score": [

       round(bleu1["bleu"], 4),

       round(bleu2["bleu"], 4),

       round(bleu3["bleu"], 4),

       round(bleu4["bleu"], 4),

       round(rouge_scores["rouge1"], 4),

       round(rouge_scores["rouge2"], 4),

       round(rouge_scores["rougeL"], 4)

   ]

})


print("\nEvaluation Results")

print(metrics_table)


metrics_table.to_csv("evaluation_metrics.csv", index=False)


# -------------------------
```

```python
# 16. Test with Sample Queries
# ------------------------
sample_queries = [
    "How can I reset my password?",
    "What payment methods do you accept?",
    "How can I track my order?"
]

print("\nSample Live Testing")
for q in sample_queries:
    print(f"\nUser Query: {q}")
    print("Generated Response:", generate_response(q))
```

app.py:

```python
import re
import streamlit as st
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM

MODEL_PATH = "./final_customer_support_model"

@st.cache_resource
def load_model():
    tokenizer = AutoTokenizer.from_pretrained(MODEL_PATH)
    model = AutoModelForSeq2SeqLM.from_pretrained(MODEL_PATH)
    return tokenizer, model

def clean_text(text):
```

```python
    text = str(text).strip().lower()

    text = re.sub(r"\s+", " ", text)

    return text


def generate_response(query, tokenizer, model):

    query = "customer_query: " + clean_text(query)

    inputs = tokenizer(query, return_tensors="pt", truncation=True, max_length=64)

    outputs = model.generate(

        **inputs,

        max_new_tokens=64,

        num_beams=4,

        early_stopping=True

    )

    return tokenizer.decode(outputs[0], skip_special_tokens=True)


st.title("Customer Support Chatbot")

st.write("Ask a customer-support question and get a generated response.")


tokenizer, model = load_model()


user_query = st.text_input("Enter your query:")


if st.button("Generate Response"):

    if user_query.strip():

        response = generate_response(user_query, tokenizer, model)

        st.subheader("Generated Response")

        st.write(response)

    else:
```

```
        st.warning("Please enter a query.")
```

OUTPUT:

Customer Support Chatbot

Ask a customer-support question and get a generated response.

Enter your query:

how can I reset my password?

Generate Response

Generated Response

to reset your password, click on the 'forgot password' link on the login page and follow the instructions to reset your password.

Customer Support Chatbot

Ask a customer-support question and get a generated response.

Enter your query:

What is your privacy policy?

Generate Response

Generated Response

our privacy policy outlines how we collect, use, and protect your personal information. please visit our privacy policy page for detailed information.

```
Sample Predictions:
                                                Query  ...                                   Generated Answer
0    Can I change my order after it has been placed?  ...  if you need to change your order, please conta...
1                    Do you offer live chat support?  ...  if you offer live chat support, please contact...
2             Do you offer a satisfaction guarantee?  ...  if you offer a satisfaction guarantee, please ...
3  Can I request a product repair or replacement ...  ...  to request a product repair or replacement if ...
4  Can I order a product that is labeled as 'limi...  ...  if you need to order a product labeled as 'lim...
5  What should I do if my package is lost or dama...  ...  if your package is lost or damaged, please con...
6                      How can I reset my password?  ...  to reset your password, click on the 'forgot p...
7                      What is your privacy policy?  ...  our privacy policy outlines how we collect, us...
8  Can I request a refund if the price drops afte...  ...  if the price of a product drops within 7 days ...
9  Can I order a product if it is listed as 'temp...  ...  if a product is listed as 'temporarily unavail...

[10 rows x 3 columns]
```

Evaluation Results

	Metric	Score
0	BLEU-1	0.5468
1	BLEU-2	0.4865
2	BLEU-3	0.4550
3	BLEU-4	0.4335
4	ROUGE-1	0.5995
5	ROUGE-2	0.4855
6	ROUGE-L	0.5427

```
Sample Live Testing

User Query: How can I reset my password?
Generated Response: to reset your password, click on the 'forgot password' link on the login page and follow the instructions to reset your password
.

User Query: What payment methods do you accept?
Generated Response: if you accept a payment method, please contact our customer support team for assistance.

User Query: How can I track my order?
Generated Response: if you need to track your order, please contact our customer support team for assistance.
```